k-Means Clustering

K-Means clustering is one of the most widely used unsupervised machine learning algorithms. It groups similar data points into k distinct, non-overlapping clusters.

The goal is to group similar data points together, where "similarity" is typically defined by a distance metric like Euclidean distance. The algorithm works by iteratively assigning data points to the nearest cluster centroid and then recalculating the centroid positions based on the newly formed clusters.

Key concepts:

- k (number of clusters): The user must specify the desired number of clusters, k, before running the algorithm.
- Centroid: The central point of a cluster, often representing the mean of the data points within that cluster.
- Distance Metric: A function that quantifies the similarity between data points, often Euclidean distance.

Core idea:

- Unsupervised Learning: k-Means is an unsupervised learning method, meaning it doesn't require labeled data (predefined categories).
- Centroid-Based: The algorithm relies on finding the "center" (centroid) of each cluster.
- Distance Minimization: The objective is to minimize the sum of squared distances between data points and their respective cluster centroids.

How it works:

- Initialization step: The algorithm starts by randomly selecting k data points as initial cluster centroids.
- Assignment step: Each data point is assigned to the nearest centroid based on a distance metric (e.g., Euclidean distance).
- Update step: The centroids of each cluster are recalculated as the mean of all data points assigned to that cluster.
- Iteration: Steps 2 and 3 are repeated until the cluster assignments no longer change, or a maximum number of iterations is reached.

In the following Sections, we implement the K-Means algorithm step-by-step using Perl and the MXNet deep learning library. This practical implementation is organized into Jupyter cells, and each code block builds toward a full clustering solution, which is ultimately visualized using Plotly.

In [1]:
```
use strict;
use warnings;
use Data::Dump qw(dump);
```

```
use sml qw(show_plot);
use AI::MXNet qw(mx nd);
IPerl->load_plugin('Chart::Plotly');
```

Data Preparation

We begin by loading the famous Iris dataset using load_csv() function from our sml module. This dataset contains 150 samples with four numerical features and one categorical class label. The class label is converted into an integer via str_column_to_int() function. Finally, the dataset is converted into an MXNet NDArray for efficient numerical operations.

In [2]:
```
my ($dataset, $header) = sml->load_csv('../data/iris.csv');
my ($lookup, $rlookup) = sml->str_column_to_int($dataset, -1);
$dataset   = nd->array($dataset);
```

Out[2]:
```
<AI::MXNet::NDArray 150x5 @cpu(0)>
```

In [3]:
```
printf "%d-%s ", $_, $header->[$_] for (0 .. $#$header);
printf "%s\n", dump $lookup, $rlookup;
```

```
0-SepalLengthCm 1-SepalWidthCm 2-PetalLengthCm 3-PetalWidthCm 4-Species (
  { "Iris-setosa" => 0, "Iris-versicolor" => 1, "Iris-virginica" => 2 },
  { "0" => "Iris-setosa", "1" => "Iris-versicolor", "2" => "Iris-virginica" },
)
```

Out[3]:
```
1
```

Let's see the first four columns (i.e., the numerical features) for clustering. A dictionary %dict is created to map column names to indices, and a sample of the feature matrix is printed for verification.

In [4]:
```
my $X = $dataset->slice(':', [0, -1]);
my $y = $dataset->slice(':', -1);
printf "%d-%s ", $_, $header->[$_] for (0 .. $#$header -1);
print $X->slice([undef, 5])->asstr;
```

```
0-SepalLengthCm 1-SepalWidthCm 2-PetalLengthCm 3-PetalWidthCm
[[5.1000 3.5000 1.4000 0.2000]
 [4.9000      3 1.4000 0.2000]
 [4.7000 3.2000 1.3000 0.2000]
 [4.6000 3.1000 1.5000 0.2000]
 [     5 3.6000 1.4000 0.2000]]
```

Out[4]:
```
1
```

Normalización de los Datos (Escalamiento Z-Score o MinMax)

Si pasamos la matriz Iris directamente con sus valores en centímetros, las variables con rangos dinámicos más grandes dominan el cálculo de la distancia euclidiana en assign_cluster.

Si normalizamos la data antes de entrenar, veremos que las curvas de tus SSEs cambian drásticamente y bajan con mayor fluidez hacia una segmentación más real.

In [5]:
```
my $X_normalized = $X;
my $minmax = sml->dataset_minmax($X_normalized);
nd->print("Matriz Min-Max por columna:", $minmax);

# normalize
sml->normalize_dataset($X_normalized, $minmax);
nd->print("Normalized:", $X_normalized->slice([0, 5]));
```

```
Matriz Min-Max por columna:
[[4.3000 7.9000]
 [     2 4.4000]
 [     1 6.9000]
 [0.1000 2.5000]]
<AI::MXNet::NDArray 4x2 @cpu(0)> float32

Normalized:
[[0.2222 0.6250 0.0678 0.0417]
 [0.1667 0.4167 0.0678 0.0417]
 [0.1111 0.5000 0.0508 0.0417]
 [0.0833 0.4583 0.0847 0.0417]
 [0.1944 0.6667 0.0678 0.0417]]
<AI::MXNet::NDArray 5x4 @cpu(0)> float32
```

Out[5]: 1

Random Initialization of Centroids

Next, we create a function named random_centroids() which initializes K centroids by randomly shuffling the dataset and selecting the first K data points. The seed is fixed to ensure reproducibility.

🔍 Purpose: Randomly initialize K centroids from the dataset.

🧠 How it works: Sets a random seed (so results are reproducible).

Shuffles all the rows in the dataset ($all_vals).

Takes the first K rows as initial centroids.

💡 Why it's important: K-Means starts by randomly placing K centroids; this function does that. Good initial centroids can help the algorithm converge faster and avoid poor local minima.

In [6]:
```
sub random_centroids{
  my ($self, $all_vals, $K) = @_;
  # Place K centroids at random locations
  my $indices   = nd->arange($all_vals->len)->shuffle();
  my $shuffled  = nd->take($all_vals, $indices, axis=>0);
  return $shuffled->slice([0, $K]);
}

sml->add_to_class('random_centroids', \&{'random_centroids'});
```

Out[6]: *sml::random_centroids

Cluster Assignment

Here, we assign each data point to the closest centroid. For each point, the index of the nearest centroid is stored by using the Euclidean distance to all centroids. This results in an array of cluster assignments.

🔍 Purpose: Assign each data point to the closest centroid.

🧠 How it works: For each data point, compute Euclidean distance to all centroids.

Choose the centroid with the minimum distance.

Collect the closest centroid index for every point.

💡 Why it's important: This is the assignment step in K-Means. Every iteration of the algorithm updates the group (cluster) each point belongs to based on proximity to centroids.

In [7]:
```
sub assign_cluster {
  my ($self, $all_vals, $centroids) = @_;

  # 1. ||x||^2 -> Suma de cuadrados a lo largo de los atributos (N, 1)
  my $data_norms = $all_vals->square()->sum(axis=>1, keepdims=>1);

  # 2. ||c||^2 -> Suma de cuadrados de los centroides (1, K)
  my $centroid_norms = $centroids->square()->sum(axis=>1)->reshape([1, -1]);

  # 3. -2 * <x, c> -> Producto punto matricial (N, D) x (D, K) = (N, K)
  my $dot_product = nd->dot($all_vals, $centroids->T);

  # 4. Combinar términos usando broadcasting: (N, 1) - 2*(N, K) + (1, K) = (N, K)
  my $dists = $data_norms - (2 * $dot_product) + $centroid_norms;

  # Retornar el índice del centroide más cercano
  return nd->argmin($dists, axis=>1);
}
sml->add_to_class('assign_cluster', \&{'assign_cluster'});
```

Out[7]:
```
*sml::assign_cluster
```

We vectorized the computation of distances using broadcasting. Instead of a loop, we compute the pairwise squared Euclidean distance matrix all at once, then find the minimum index along axis 1.

Centroid Update

Once points are assigned to clusters, new centroids are calculated by averaging the features of points within each cluster. A boolean mask selects the relevant indices, and the mean is computed across the selected data points.

🔍 Purpose: Recompute the centroids by taking the mean of all assigned points for each cluster.

🧠 How it works: For each cluster i, selects points assigned to it.

Calculates the mean (average) across each feature dimension.

Concatenates all new centroids into a matrix.

💡 Why it's important: This is the update step in K-Means. It adjusts the centroid positions so they reflect the center of their assigned data points.

In [8]:
```
sub new_centroids{
  my ($self, $all_vals, $centroids, $assignments, $K) = @_;
  my $one_hot = nd->one_hot($assignments, $K);                # (N, K)
  my $counts  = nd->sum($one_hot, axis=>0)->reshape([$K, 1]); # Forzamos (K, 1) para
  my $sums    = nd->dot($one_hot->T, $all_vals);              # (K, D)
  return $sums / $counts; # means: (K, D) / (K, 1) -> Operación directa limpia
}

sml->add_to_class('new_centroids', \&{'new_centroids'});
```

Out[8]:
```
*sml::new_centroids
```

Instead of looping over clusters, we use one-hot encoding of assignments and matrix multiplication to compute the cluster-wise sums. Dividing these sums by the counts yields the new centroids.

Sum of Squared Errors computation

The SSE function quantifies the clustering error by summing the squared distances from each point to its assigned centroid. It computes the assigned centroid for each data point and calculates squared errors in a fully parallelized way. This metric is crucial for convergence checking.

🔍 Purpose: Calculate the total error of the current clustering, a metric to monitor convergence.

🧠 How it works: For each point, subtract its assigned centroid to compute the distance.

Compute squared norm (Euclidean distance squared).

Sum all squared distances to get SSE.

💡 Why it's important: The SSE value helps determine whether the algorithm is converging — lower SSE implies better compactness of clusters.

In [9]:
```
sub sse{
  my ($self, $all_vals, $assignments, $centroids)= @_;
  my $centroid = nd->take($centroids, $assignments, axis=>0);
  my $errors   = nd->norm($all_vals - $centroid, axis=>1);
  return $errors->power(2)->sum();
}

sml->add_to_class('sse', \&{'sse'});
```

Out[9]:
```
*sml::sse
```

K-Means main loop

The next subroutine ties everything together. It runs the K-Means algorithm in a loop:

- Randomly initializes centroids.
- Reassigns clusters.
- Updates centroids.
- Computes SSE.

The loop stops when the change in SSE falls below a tolerance threshold or when the maximum number of iterations is reached.

🔍 Purpose: Runs the complete K-Means algorithm until it converges.

🧠 How it works: Initializes centroids.

Iteratively:

Assigns each point to the nearest centroid.

Updates centroids based on assignments.

Computes the SSE to monitor change.

Stops when:

SSE change is small (< tol), or

Maximum number of iterations reached.

💡 Why it's important: This is the core driver of the K-Means algorithm. It handles convergence logic and returns final assignments, centroids, SSE progression, and number of iterations used.

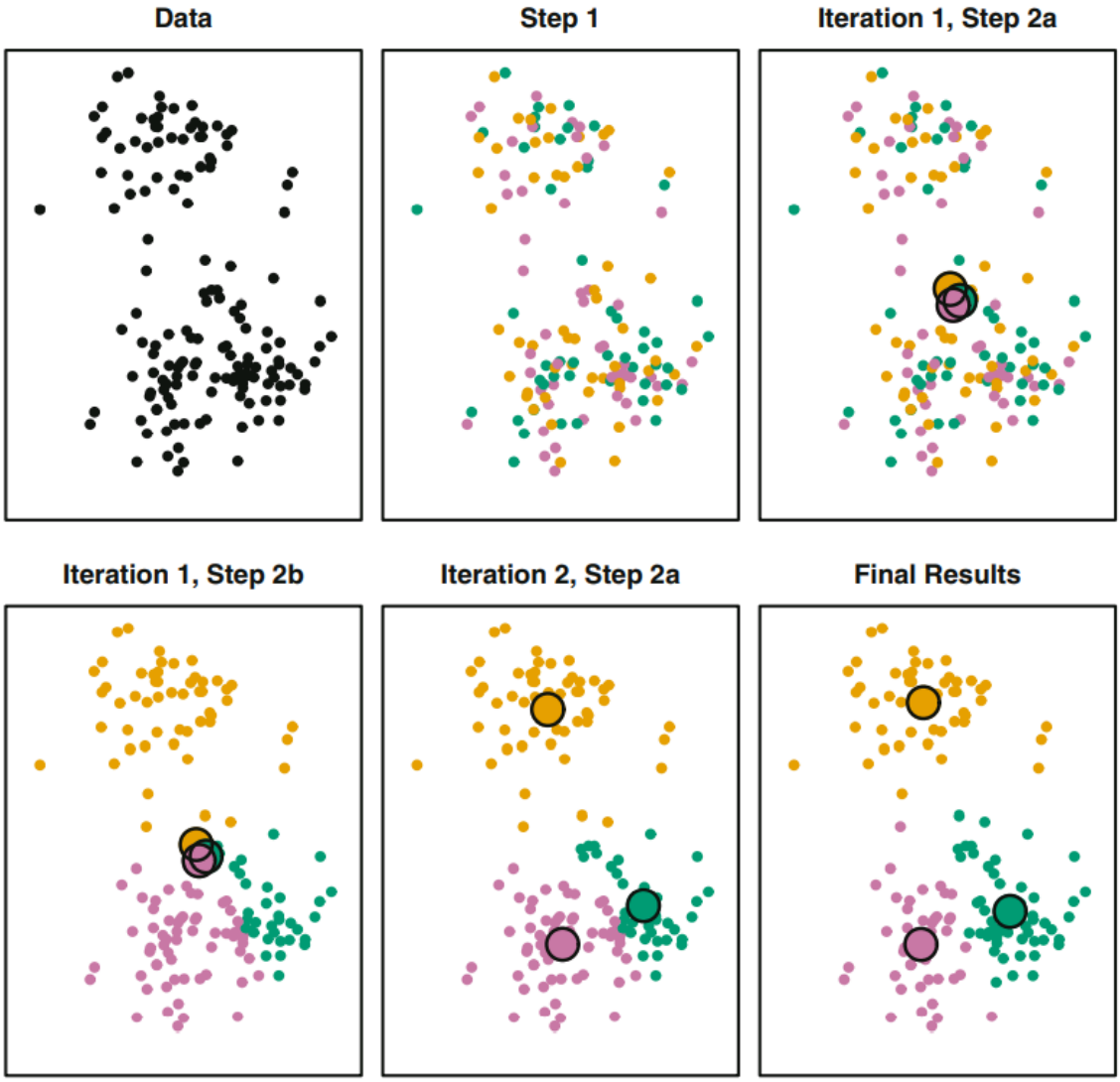


Fig. 1. K-Means.

In [10]:
```
sub kmeans_clustering {
  my ($self, $all_vals, $K, $max_iter) = @_;
  $max_iter //= 20;

  my $centroids = sml->random_centroids($all_vals, $K);
  my $assignments;
  my @all_sse; # Aquí guardaremos el histórico de tensores SSE

  for (1 .. $max_iter) {
    $assignments = sml->assign_cluster($all_vals, $centroids);
    $centroids   = sml->new_centroids($all_vals, $centroids, $assignments, $K);

    # 1. Calculamos el SSE en cada iteración como un tensor (¡Sin usar ->asscalar!)
    my $sse = sml->sse($all_vals, $assignments, $centroids);
    push @all_sse, $sse;
  }

  # 2. Concatenamos todos los SSEs acumulados en un único tensor de MXNet al salir de
  # Esto mantiene la ejecución asíncrona y eficiente.
  my $historical_sse = nd->concat(@all_sse, dim=>0);

  return $assignments, $centroids, $historical_sse;
```

```
}

sml->add_to_class('kmeans_clustering', \&{'kmeans_clustering'});
```

Out[10]:
```
*sml::kmeans_clustering
```

Running the Clustering

Next, we execute the clustering algorithm with K=3, which corresponds to the same number of species in the Iris dataset. The final assignments, centroids, error progression, and iteration count are returned.

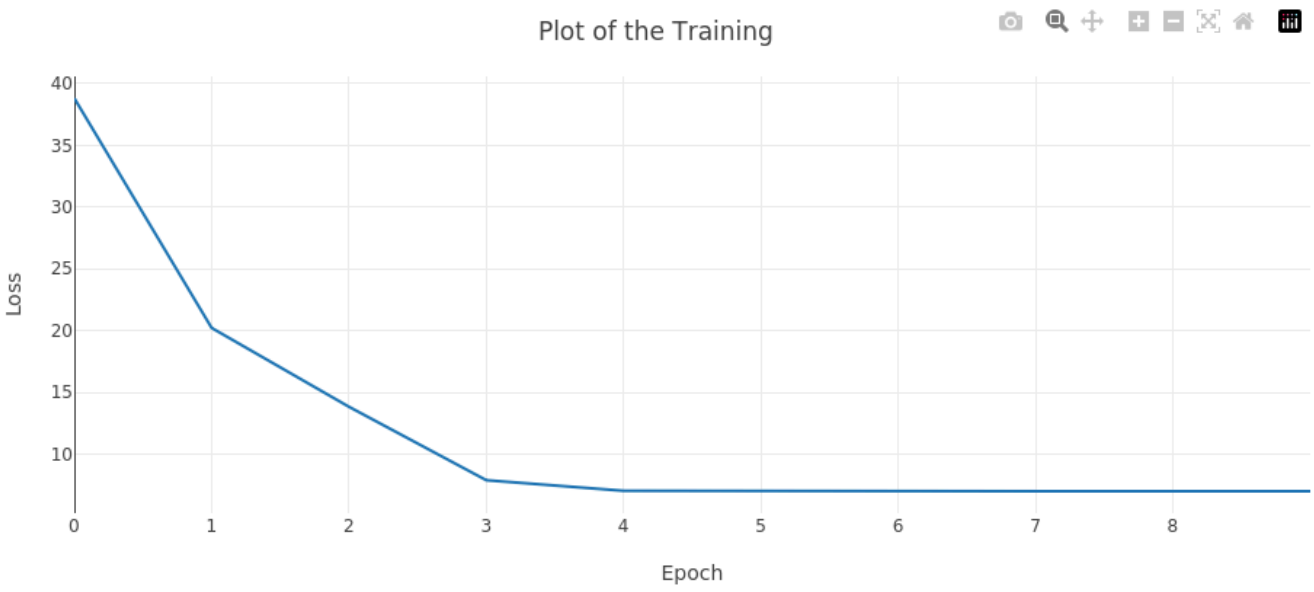
In [11]:
```
mx->random->seed(576);
my $K = 3;
my ($assignments, $centroids, $all_sse) =
    sml->kmeans_clustering($X_normalized, $K, 10);
nd->print($all_sse);
```

```
[38.8079 20.2181 13.8353  7.8917  7.0352  6.9981  6.9981  6.9981  6.9981  6.9981]
<AI::MXNet::NDArray 10 @cpu(0)> float32
```

Out[11]:
```
1
```

In [12]:
```
my $trace = new Chart::Plotly::Trace::Scatter(x => nd->arange($all_sse->len)->asarray
                                              y => $all_sse->asarray,
                                              name => 'Train',
                                              mode => 'lines');

my $layout = {title => {text => 'Plot of the Training'},
              xaxis => {title => 'Epoch'}, yaxis => {title => 'Loss'},
              width  => 900, height => 400,
              margin => { l => 50, r => 0, t => 50, b => 50 }};

my $plot = new Chart::Plotly::Plot(traces => [$trace],
                                   layout => $layout);

IPerl->display($plot);
```

In [13]:
```
sml->embedplot($plot, width=>900, height=>450);
```



El problema del mapeo de etiquetas

K-Means no sabe qué es Setosa, Versicolor o Virginica; solo agrupa los datos como Clúster 0, Clúster 1 o Clúster 2 basándose en distancias.

Antes de evaluar el desempeño del k-means, debemos asegurarnos de que el índice de la columna corresponda con la clase real de la fila.

In [14]:

```
print $assignments->aspdl;
```

```
[2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 1 0 0 1 1 0
 0 0 0 0 1 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1]
```

Out[14]: 1

In [15]:

```
print $y->aspdl;
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
```

Out[15]: 1

In [16]:

```
print $centroids->asstr;
# [[0.7073 0.4509 0.7970 0.8248]
#  [0.4413 0.3074 0.5757 0.5492]
#  [0.1961 0.5908 0.0786 0.0600]]
```

```
[[0.7073 0.4509 0.7970 0.8248]
 [0.4413 0.3074 0.5757 0.5492]
 [0.1961 0.5908 0.0786 0.0600]]
```

Out[16]: 1

In [17]:

```
my $sort_idx = nd->array([2, 1, 0]);
my $sorted_centroids = nd->take($centroids, $sort_idx);
nd->print($sorted_centroids);
```

```
[[0.1961 0.5908 0.0786 0.0600]
 [0.4413 0.3074 0.5757 0.5492]
 [0.7073 0.4509 0.7970 0.8248]]
<AI::MXNet::NDArray 3x4 @cpu(0)> float32
```

Out[17]: 1

In [18]:

```
sub evaluate_centroids {
  my ($self, $test_vals, $centroids) = @_;

  # 1. Asignar los puntos de prueba al centroide más cercano (usando tu función optim
  my $test_assignments = $self->assign_cluster($test_vals, $centroids);

  # 2. Calcular el SSE en el conjunto de prueba utilizando los centroides fijos
  my $test_sse = $self->sse($test_vals, $test_assignments, $centroids);

  # Retornamos las etiquetas asignadas a los datos de prueba y el error escalar
  return $test_assignments, $test_sse;
}

sml->add_to_class('evaluate_centroids', \&{'evaluate_centroids'});
```

Out[18]: *sml::evaluate_centroids

In [19]:

```
my ($test_assignments, $test_sse) =
    sml->evaluate_centroids($X_normalized, $sorted_centroids);
```

Out[19]: <AI::MXNet::NDArray 150 @cpu(0)><AI::MXNet::NDArray 1 @cpu(0)>

In [20]:
```
nd->print($test_assignments);
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 1 2 2 2 2 2 2 1 2 2 2 2 2 1 2 1 2 1 2 2 1 1 2
 2 2 2 2 1 1 2 2 2 1 2 2 2 1 2 2 2 1 2 2 1]
<AI::MXNet::NDArray 150 @cpu(0)> float32
```

Out[20]: 1

In [21]:
```
my $matrix = sml->confusion_matrix($y, $test_assignments);
print $matrix->asstr;
```

```
[[50  0  0]
 [ 0 47  3]
 [ 0 14 36]]
```

Out[21]: 1

Solución del problema del mapeo de etiquetas

Al observar la matriz de confusión:

Clúster 0 (Fila 1): Agrupó perfectamente 50 elementos de una clase sin errores.

Clúster 1 (Fila 2): Agrupó 47 de una clase y 3 de otra.

Clúster 2 (Fila 3): Agrupó 36 de una clase y se le colaron 14 de otra.

Los datos ya están alineados (es decir, la diagonal principal representa los verdaderos positivos de cada clase), puedes proceder directamente con la estrategia One-vs-Rest (Uno contra todos) para reutilizar las funciones binarias de evaluación del modelo.

In [22]:
```
sub multiclass_perf_metrics {
    my ($self, $actual, $predicted) = @_;

    # 1. Obtener la matriz de confusión global usando tu método existente
    # Si viene en formato NDArray, $matrix será un NDArray, de lo contrario un array
    my $matrix = $self->confusion_matrix($actual, $predicted);

    my $is_tensor = (ref($matrix) =~ /^AI::MXNet::NDArray(?:::Slice)?$/);
    my $num_classes = $is_tensor ? $matrix->shape->[0] : scalar(@$matrix);

    my %report = ();
    my ($global_tp, $total_elements) = (0, 0);

    # 2. Iterar sobre cada clase bajo la estrategia One-vs-Rest
    for my $i (0 .. $num_classes - 1) {
        my ($tp, $fp, $fn, $tn) = (0, 0, 0, 0);

        if ($is_tensor) {
            # Extraer métricas binarias usando indexación nativa de MXNet
            $tp = $matrix->at($i, $i)->asscalar;

            # FP: Suma de la columna 'i' menos el verdadero positivo
            $fp = nd->sum($matrix->slice(':', $i))->asscalar - $tp;

            # FN: Suma de la fila 'i' menos el verdadero positivo
            $fn = nd->sum($matrix->slice($i, ':'))->asscalar - $tp;

            # TN: Suma total de toda la matriz menos los otros tres componentes
            $tn = nd->sum($matrix)->asscalar - ($tp + $fp + $fn);
```

```
        } else {
            # Extraer métricas usando desreferenciación estándar de Perl
            $tp = $matrix->[$i][$i];

            for my $row (0 .. $num_classes - 1) {
                for my $col (0 .. $num_classes - 1) {
                    if ($row == $i && $col == $i) { next; }
                    elsif ($row == $i) { $fn += $matrix->[$row][$col]; }
                    elsif ($col == $i) { $fp += $matrix->[$row][$col]; }
                    else               { $tn += $matrix->[$row][$col]; }
                }
            }
        }

        # 3. Calcular métricas clásicas de clasificación para la clase actual
        my $precision = ($tp + $fp) > 0 ? ($tp / ($tp + $fp)) : 0;
        my $recall    = ($tp + $fn) > 0 ? ($tp / ($tp + $fn)) : 0;
        my $f1_score  = ($precision + $recall) > 0 ?
                        (2 * ($precision * $recall) / ($precision + $recall)) : 0;

        # Guardar resultados de la clase
        $report{"clase_$i"} = {
            precision => sprintf('%0.4f', $precision),
            recall    => sprintf('%0.4f', $recall),
            f1_score  => sprintf('%0.4f', $f1_score),
            support   => $tp + $fn
        };

        $global_tp      += $tp;
        $total_elements += ($tp + $fn);
    }

    # 4. Calcular métrica Micro-Average Global (Equivalente al Accuracy General)
    $report{global_accuracy} = sprintf('%0.4f', $total_elements > 0 ?
                               ($global_tp / $total_elements) * 100 : 0);

    return \%report;
}

sml->add_to_class('multiclass_perf_metrics', \&{'multiclass_perf_metrics'});
```

Out[22]: *sml::multiclass_perf_metrics

In [23]:
```
sub print_classification_report {
    my ($self, $report) = @_;

    print "\n" . "=" x 58 . "\n";
    print "                CLASSIFICATION REPORT\n";
    print "=" x 58 . "\n";
    # Encabezados de las columnas con anchos fijos
    printf "%-12s %10s %10s %10s %10s\n", "Clase", "Precision", "Recall", "F1-Score",
    print "-" x 58 . "\n";

    # Iterar sobre las clases ordenadas (clase_0, clase_1, clase_2)
    for my $key (sort grep { /^clase_/ } keys %$report) {
        my $metrics = $report->{$key};
        # Formatear el nombre de la clase para mostrar solo el ID o el string limpio
        my $class_name = ucfirst($key); $class_name =~ s/_/ /;

        printf "%-12s %10.4f %10.4f %10.4f %10d\n",
            $class_name,
            $metrics->{precision},
            $metrics->{recall},
            $metrics->{f1_score},
            $metrics->{support};
```

```
    }
    print "-" x 58 . "\n";

    # Métrica Global
    printf "%-47s %10.2f%%\n", "Accuracy Global:", $report->{global_accuracy};
    print "=" x 58 . "\n\n";
}

# Inyectar o ejecutar directamente:
sml->add_to_class('print_classification_report', \&{'print_classification_report'});
```

Out[23]:
```
*sml::print_classification_report
```

In [24]:
```
my $reporte_metricas = sml->multiclass_perf_metrics($y, $test_assignments);
```

Out[24]:
```
HASH(0x5568d7b03088)
```

In [25]:
```
sml->print_classification_report($reporte_metricas);
```

```
============================================================
                CLASSIFICATION REPORT
============================================================
Clase         Precision     Recall   F1-Score    Support
------------------------------------------------------------
Clase 0          1.0000     1.0000     1.0000         50
Clase 1          0.7705     0.9400     0.8468         50
Clase 2          0.9231     0.7200     0.8090         50
------------------------------------------------------------
Accuracy Global:                                   88.67%
============================================================
```

Out[25]:
```
1
```

Obtención de Probabilidades mediante Softmax sobre Distancias

Para poder graficar curvas ROC con la función perf_metrics() definida en la librería sml, necesitamos pasarle un vector con las probabilidades continuas estimadas de la clase positiva (un rango entre 0.0 y 1.0) para cada una de las muestras, en lugar de las asignaciones discretas de clústeres (0, 1, 2).

Dado que K-Means es un algoritmo de agrupamiento duro (hard clustering) basado en distancias euclidianas muertas, por defecto no devuelve probabilidades. Sin embargo, podemos transformar el modelo en un clasificador probabilístico utilizando la distancia inversa a los centroides mediante la función de activación Softmax.

A continuación, implementaremos metodológicamente esta vectorización usando AI::MXNet para obtener esa matriz de probabilidades continuas:

La idea es calcular las distancias de cada punto a todos los centroides, usando la función multiclass_perf_metrics(). Como buscamos una probabilidad, una distancia menor debe representar una probabilidad mayor. Por tanto, para transformar las distancias euclidianas del algoritmo K-Means en probabilidades continuas, se utiliza la función de activación Softmax aplicada al inverso de las distancias:

$$P(C_k|x_i) = \frac{e^{-\|x_i - c_k\|^2}}{\sum_{j=1}^{K} e^{-\|x_i - c_j\|^2}}$$

donde:

- $P(C_k|x_i)$: Probabilidad a posteriori de que la muestra u observación x_i pertenezca al clúster o clase k. Este valor estará estrictamente acotado en el rango $(0, 1)$ y la suma de estas probabilidades para todas las clases posibles K será exactamente igual a 1.
- x_i: Vector de características de la i-ésima observación (en tu caso, un punto del dataset Iris con dimensiones $1 \times D$).
- c_k: Vector que representa las coordenadas del centroide del clúster k (dimensión $1 \times D$).
- $||x_i - c_k||^2$: Distancia euclidiana al cuadrado entre la observación x_i y el centroide c_k. Actúa como la medida de disimilitud.
- $e^{-||x_i - c_k||^2}$: Exponencial natural aplicada al **negativo** de la distancia. Al introducir el signo negativo, se invierte la relación lógica: una distancia muy pequeña (cercana a 0) se convierte en un valor cercano a $e^0 = 1$ (alta probabilidad), mientras que una distancia muy grande tiende a $e^{-\infty} = 0$ (baja probabilidad).
- $\sum_{j=1}^{K} e^{-||x_i - c_j||^2}$: Término de normalización (denominador). Es la suma de las exponenciales de las distancias negativas hacia **todos** los K centroides del sistema. Su función es escalar los valores del numerador para que el resultado final cumpla con los axiomas de una distribución de probabilidad válida.

In [26]:
```
sub predict_proba {
    my ($self, $all_vals, $centroids) = @_;

    # 1. Calcular las distancias al cuadrado (N, K) usando tu álgebra del trinomio
    my $data_norms = $all_vals->square()->sum(axis=>1, keepdims=>1);
    my $centroid_norms = $centroids->square()->sum(axis=>1)->reshape([1, -1]);
    my $dot_product = nd->dot($all_vals, $centroids->T);
    my $dists = $data_norms - (2 * $dot_product) + $centroid_norms;

    # 2. Convertir distancias a probabilidades usando Softmax sobre el inverso (-dist
    # axis=>1 asegura que las probabilidades de cada fila sumen 1.0
    my $probabilities = nd->softmax(-$dists, axis=>1);

    return $probabilities; # Retorna un NDArray de dimensiones (N, K)
}

sml->add_to_class('predict_proba', \&{'predict_proba'});
```

Out[26]:
```
*sml::predict_proba
```

A continuación definimos la lógica de una función compute_roc_vectors, que automatiza la estrategia One-vs-Rest (OvR) aislando dinámicamente la clase objetivo (etiqueta positiva) y barriendo el espacio de umbrales desde 0.0 hasta 1.0 de forma totalmente parametrizada.

In [27]:
```
sub compute_roc_vectors {
    my ($self, $actual, $probabilities, $positive_class, %args) = @_;

    # Parámetro opcional para controlar la resolución del barrido de umbrales
    my $steps = $args{steps} // 100; # por defecto 100 pasos
    my @thresholds = map { $_ / $steps } (0 .. $steps);

    my (@fpr_points, @tpr_points);
    my $actual_binary;
    my $prob_target;

    # 1. Detectar si las entradas son tensores de MXNet o estructuras nativas de Perl
    if (ref($actual) =~ /^AI::MXNet::NDArray(?:::Slice)?$/) {

        # Crear vector binario: 1.0 para la clase positiva, 0.0 para el resto (One-vs
        $actual_binary = ($actual == $positive_class)->astype('float32');
```

```
        # Extraer únicamente la columna de la probabilidad continua de la clase posit
        $prob_target = $probabilities->slice(':', $positive_class);

    } else {
        # Si son arreglos de Perl estándar (conversión defensiva en caso de flujos mi
        $actual_binary = [ map { $_ == $positive_class ? 1 : 0 } @$actual ];
        $prob_target   = [ map { $_->[$positive_class] } @$probabilities ];
    }

    # 2. Barrido iterativo sobre el espacio de umbrales utilizando tu función binaria
    for my $th (@thresholds) {
        my ($fpr, $tpr, $matrix) = $self->perf_metrics(
            $actual_binary,
            $prob_target,
            $th,
            positive_class => 1
        );

        push @fpr_points, $fpr;
        push @tpr_points, $tpr;
    }

    # 3. Invertir los vectores para asegurar el orden ascendente
    # estándar en el eje X de la curva ROC
    @fpr_points = reverse @fpr_points;
    @tpr_points = reverse @tpr_points;

    # Retornar las coordenadas listas para graficar y el cálculo de área bajo la curv
    my $auc = $self->trapz(\@fpr_points, \@tpr_points);

    return \@fpr_points, \@tpr_points, $auc;
}

sml->add_to_class('compute_roc_vectors', \&{'compute_roc_vectors'});
```

Out[27]:
```
*sml::compute_roc_vectors
```

In [28]:
```
# Supongamos que ya tienes los centroides entrenados y tus datos cargados
my $probs = sml->predict_proba($X_normalized, $sorted_centroids);
```

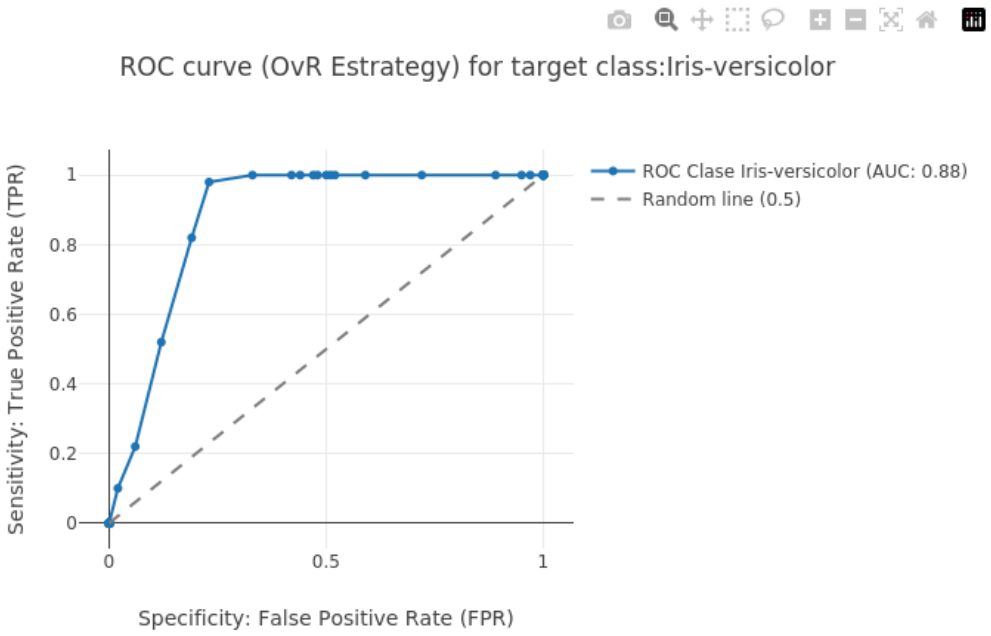
Out[28]:
```
<AI::MXNet::NDArray 150x3 @cpu(0)>
```

In [29]:
```
# Elegimos una clase a evaluar
my $target_class = 1;

# Llamada a la nueva función compute_roc_vectors()
my ($fpr, $tpr, $auc) = sml->compute_roc_vectors($y, $probs, $target_class, steps =>

print "--- ROC metrics for the class $rlookup->{$target_class} ---\n";
print "Area below the curve (AUC): $auc\n";

my $roc_trace = new Chart::Plotly::Trace::Scatter(
    x    => $fpr,
    y    => $tpr,
    mode => 'lines+markers',
    name => "ROC Clase $rlookup->{$target_class} (AUC: $auc)"
);

my $diagonal = new Chart::Plotly::Trace::Scatter(
    x    => [0, 1], y => [0, 1], mode => 'lines',
    line => { dash => 'dash', color => 'gray' }, name => 'Random line (0.5)'
);

my $plot = new Chart::Plotly::Plot(
    traces => [$roc_trace, $diagonal],
```

```
    layout => {
        title  => "ROC curve (OvR Estrategy) for target class:" .
                  "$rlookup->{$target_class}",
        xaxis => { title => 'Specificity: False Positive Rate (FPR)' },
        yaxis => { title => 'Sensitivity: True Positive Rate (TPR)' }
    }
);

IPerl->display($plot);
```

```
--- ROC metrics for the class Iris-versicolor ---
Area below the curve (AUC): 0.88
```

In [30]:
```
sml->embedplot($plot, width=>900, height=>450);
```



Configuración del experimento para optimizar la búsqueda de la semilla

A continuación definimos un experimento para buscar la semilla en donde se busque la semilla con la máxima reducción de error.

In [31]:
```
my $K = 3;
my $max_iter = 10;
my $total_seeds_to_test = 100;

my $max_delta = -1;
my $best_seed = undef;
my ($best_sse_start, $best_sse_end);

print "Iniciando búsqueda de la peor inicialización (Máxima Delta SSE)...\n";
print "-----------------------------------------------------------------\n";
my $start = 0; # Puedes aumentar el inicio para explorar nuevas semillas

for my $current_seed (1 + $start .. $total_seeds_to_test + $start) {
    # 1. Fijar la semilla actual de forma determinista para MXNet
    mx->random->seed($current_seed);

    # 2. Ejecutar tu algoritmo optimizado con el retorno histórico de SSEs
    my ($assignments, $centroids, $all_sse) = sml->kmeans_clustering($X, $K, $max_ite

    # 3. Extraer el primer y el último SSE usando operaciones nativas y asscalar
    my $sse_inicio = $all_sse->slice([0])->asscalar;
    my $sse_final  = $all_sse->slice([-1])->asscalar;
```

```
    # 4. Calcular cuánta pérdida se redujo
    my $delta_sse = $sse_inicio - $sse_final;

    # Control de logs opcional por cada semilla para monitorear el progreso
    # printf "Semilla %02d -> Inicio: %.4f | Final: %.4f | Delta: %.4f\n",
    # $current_seed, $sse_inicio, $sse_final, $delta_sse;

    # 5. Evaluar si es la máxima diferencia encontrada hasta el momento
    if ($delta_sse > $max_delta) {
        $max_delta      = $delta_sse;
        $best_seed      = $current_seed;
        $best_sse_start = $sse_inicio;
        $best_sse_end   = $sse_final;
    }
}

print "----------------------------------------------------------------------\n";
print "¡Búsqueda completada con éxito!\n";
print "=> La semilla con la máxima reducción de error es: Semilla $best_seed\n";
print "   SSE Inicial (Peor caso): $best_sse_start\n";
print "   SSE Final (Convergencia): $best_sse_end\n";
print "   Variación del Error (Delta): $max_delta\n";
```

```
Iniciando búsqueda de la peor inicialización (Máxima Delta SSE)...
----------------------------------------------------------------------
----------------------------------------------------------------------
¡Búsqueda completada con éxito!
=> La semilla con la máxima reducción de error es: Semilla 82
   SSE Inicial (Peor caso): 38.013614654541
   SSE Final (Convergencia): 7.13864755630493
   Variación del Error (Delta): 30.8749670982361
```

Out[31]: 1

Visualization of the clusters vs the actual labels.

Finally, we visualize the result using Plotly. Each data point is colored by its assigned cluster, and centroids are shown with a red cross. The visualization uses Sepal Length and Petal Length as axes for clarity.

We plot:

- The data points colored by cluster.
- The centroids marked in red crosses.

We use Chart::Plotly for interactive visualization. This helps visually inspect how well the clustering performed.

In [32]:
```
my ($X0, $X1, $X2, $X3) = @{$X_normalized->T};

my $color_scale = [
    [0,   'green'],
    [0.5, 'purple'],
    [1,   'orange']
];
```

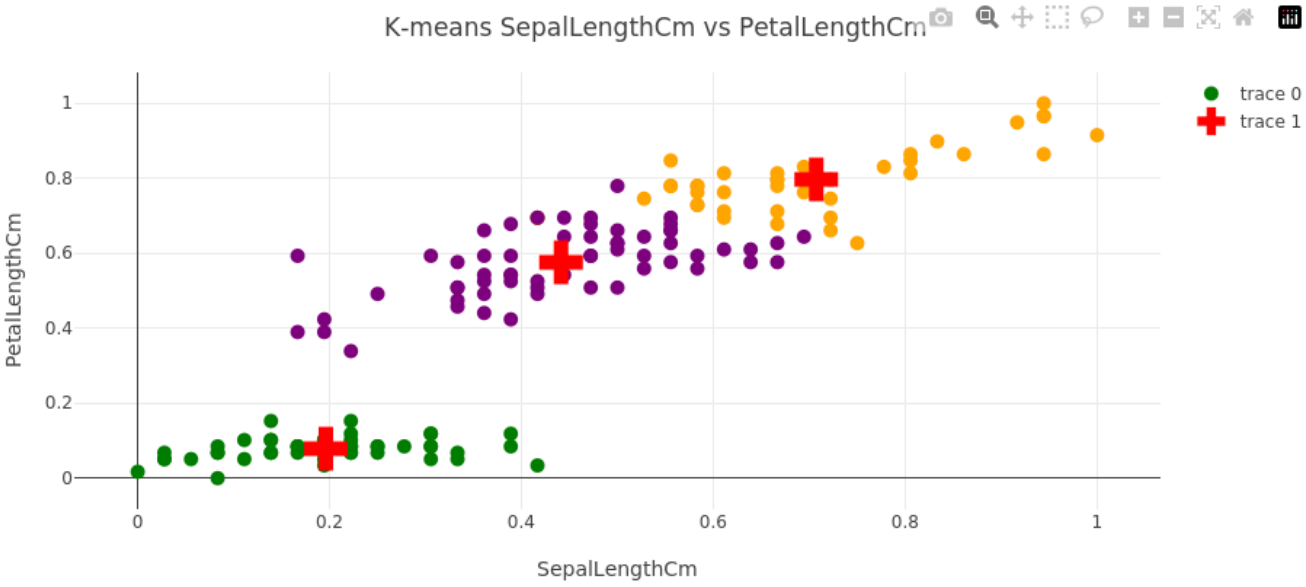
Out[32]: ARRAY(0x5568d7af16b8)

In [33]:
```
# Now, we take any two columns from the dataset to visually
# compare with their counterpart labels.
```

```
my $traces = new Chart::Plotly::Trace::Scatter(
    x      => $X0->aspdl,
    y      => $X2->aspdl,
    mode   => 'markers',
    marker => {color      => $test_assignments->aspdl,
               colorscale => $color_scale,
               size=>10},
);

my $centroids_trace = new Chart::Plotly::Trace::Scatter(
    x      => $centroids->slice(':', 0)->aspdl,
    y      => $centroids->slice(':', 2)->aspdl,
    mode   => 'markers',
    marker => {symbol => 'cross', color=>'red',  size=>25},
);

my $layout = {title => {text => sprintf('K-means %s vs %s', @$header[0, 2])},
              xaxis => {title => $header->[0]}, yaxis => {title => $header->[2]},
              width  => 900, height => 400,
              margin => { l => 50, r => 0, t => 50, b => 50 }};

my $plot = new Chart::Plotly::Plot(traces=>[$traces, $centroids_trace],
                                   layout=>$layout);

IPerl->display($plot);
```

In [34]:

```
sml->embedplot($plot, width=>900, height=>450);
```

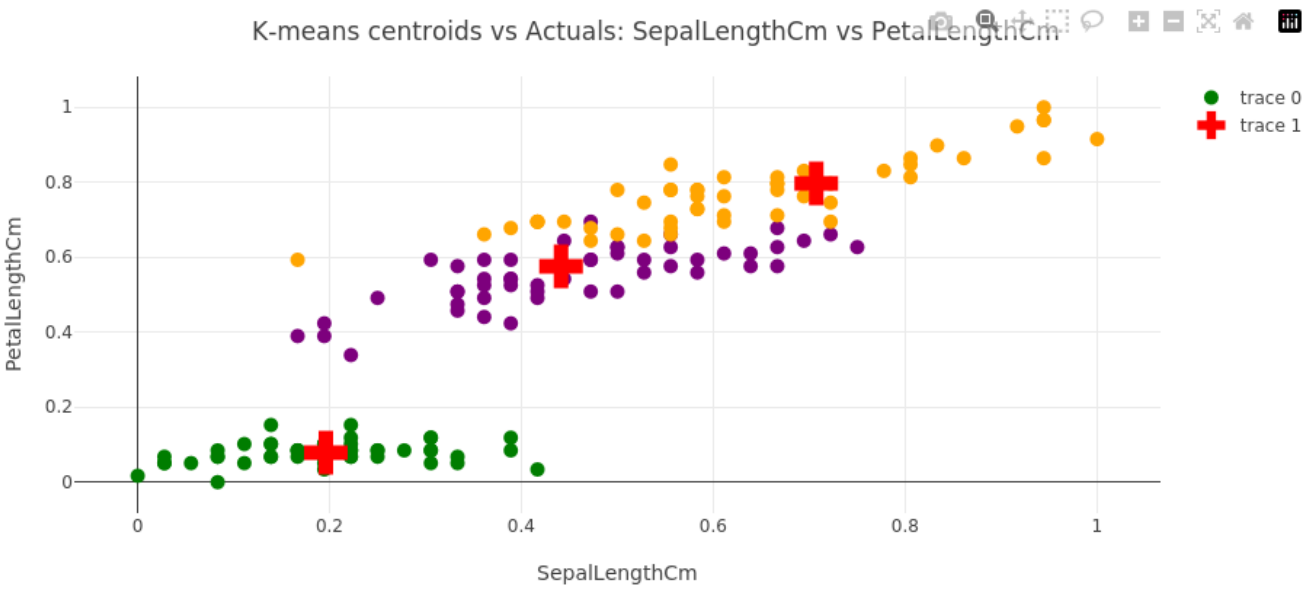


In [35]:

```
my $traces = new Chart::Plotly::Trace::Scatter(
    x      => $X0->aspdl,
    y      => $X2->aspdl,
    mode   => 'markers',
    marker => {color      => $y->aspdl,
               colorscale => $color_scale,
               size=>10},
);

my $centroids_trace = new Chart::Plotly::Trace::Scatter(
    x      => $centroids->slice(':', 0)->aspdl,
    y      => $centroids->slice(':', 2)->aspdl,
    mode   => 'markers',
    marker => {symbol => 'cross', color=>'red',  size=>25},
);

my $layout = {title => {text => sprintf('K-means centroids vs Actuals: %s vs %s',
```

```
                        @$header[0, 2])},
              xaxis => {title => $header->[0]}, yaxis => {title => $header->[2]},
              width  => 900, height => 400,
              margin => { l => 50, r => 0, t => 50, b => 50 }};

my $plot = new Chart::Plotly::Plot(traces=>[$traces, $centroids_trace],
                                   layout=>$layout);

IPerl->display($plot);
```

In [36]:
```
sml->embedplot($plot, width=>900, height=>450);
```



Applications:

- Market Segmentation: Grouping customers based on purchasing behavior.
- Image Segmentation: Dividing an image into regions based on pixel similarity.
- Document Clustering: Grouping documents based on topic or content.
- Anomaly Detection: Identifying data points that deviate significantly from their cluster.

Strengths:

- Simplicity and Speed: Relatively easy to implement and computationally efficient, especially for large datasets.
- Wide Applicability: Can be used in various domains and with different types of data.
- Interpretability: Centroids can provide insights into the characteristics of each cluster.

Limitations:

- Sensitivity to Initialization: The algorithm's results can be sensitive to the initial choice of centroids.
- Assumes Spherical Clusters: May not perform well with non-spherical or irregularly shaped clusters.
- Requires Choice of k: Requires the user to predefine the number of clusters, which can be challenging.
- Scalability: While generally fast, performance can degrade with extremely large datasets.

Conclusion

This chapter presented a complete K-Means clustering implementation in Perl using MXNet. We built the algorithm from scratch, covering data loading, centroid initialization, cluster assignment, centroid updates, convergence monitoring via SSE, and final visualization. This modular and interpretable approach helps solidify understanding of the algorithm's mechanics while providing a foundation for more advanced clustering techniques in future chapters.